

Assignment: Algorithm Implementation: Hamming Distance

林柏翰
1113405021
國立澎湖科技大學

Abstract—本報告聚焦於資訊工程中重要的字串比對演算法：漢明距離 (Hamming Distance)。實驗實作了線性掃描比對演算法，透過計算兩個等長序列之間相異的位元數，驗證其 $O(N)$ 的時間複雜度優勢。

Index Terms—Hamming Distance, String Comparison, Time Complexity, Algorithms, Java.

I. INTRODUCTION

在資料處理與編碼理論中，特徵提取與序列比對是核心技術 [cite: 115]。本作業實作了經典的「漢明距離」演算法，其主要用於計算兩個等長序列（如二進位數據或文字字串）之間的差異位元數。此概念不僅可用於錯誤偵測 (Error Detection)，也在分類演算法與資料庫比對中扮演重要角色。

II. METHODOLOGY

漢明距離的運作邏輯為逐一比對兩個等長字串對應位置的字元 [cite: 116]。若字元不同，則距離變數加一。其核心需依賴一個迴圈進行單次線性掃描：

- **時間複雜度:** $O(N)$ ，其中 N 為字串長度。演算法精準遍歷整個字串一次。
- **空間複雜度:** $O(1)$ ，僅需常數空間儲存一個用來計數的整數變數，無需額外配置記憶體陣列。

III. EXPERIMENTAL RESULTS

本實驗使用 Java 進行實作 [cite: 117, 119]。給定兩組測試資料，第一組為二進位字串 1011101 與 1001001，程式成功比對出兩者有兩個位元不同，輸出距離為 2 [cite: 117]；第二組為英文字串 toned 與 roses，程式成功辨識出 3 個不同的字元，距離為 3 [cite: 119]。終端機輸出結果如圖 1 所示。

```
--- Assignment 2: Hamming Distance ---
Distance of (1011101) and (1001001) is: 2
Distance between (toned) and (roses) is: 3

[Time Complexity]
Time Complexity: O(N), where N is the length of the string/binary data.
Space Complexity: O(1)
```

Fig. 1. 漢明距離 (Hamming Distance) 程式執行結果與時間複雜度。

IV. CONCLUSION

本次實驗成功實作了漢明距離演算法。透過 Java 程式的實作與終端機結果的驗證，確認了線性掃描在處理等長序列比對時的精確性與 $O(N)$ 運算效率，為進階的編碼理論打下良好的實作基礎。